When the Interval Is Smaller Than the Instrument: Two Ways Streaming Latency Benchmarks Fail on Sub-Millisecond Paths

Gustavo Pedro Ricou

***Abstract*—Streaming brokers are compared on sub-millisecond paths with instruments whose timescales exceed the intervals they report. One ratio governs what follows: an instrument timescale over the interval measured. Two consequences of that ratio exceeding one are invisible in the output. First, a measured latency can come out negative, because one of two stamps is written by a thread that must first be scheduled, and that stall can outlast the delivery timed. A sign check rejected $1{,}321$ of $2{,}266$ runs. Across $738{,}730$ events the acknowledgment-referenced span inverts $62{,}264$ times while the send-referenced span never does, on one clock by construction, so the cause is scheduling and not synchronization; real-time priority at fixed utilization collapses it 7–$80\times$. Second, the most widely used cross-broker benchmark admits an *end-to-end* sample only when a millisecond-quantized difference is positive and counts none of what it drops, while measuring its same-process span with a nanosecond clock, unfiltered: over 71 embedded-mode cells reporting a median on the millisecond grid it computed that median from 0.36% to 100% of the samples taken, at a retained fraction fixed by the send-interval-to-quantum ratio. The remedy is one line: dither the send instant and publish the retention rate.**

***Index Terms*—distributed systems; measurement techniques; Measurement, evaluation, modeling, simulation of multiple-processor systems; message passing; performance attributes; scheduling and task partitioning.**

I. Introduction

STATED plainly, this paper reports two ways a latency benchmark can be wrong about its own measurements, and one number that governs both.

A benchmark times a message by stamping a clock when it is sent and again when it arrives, then subtracting. Both stamps are written by software that must first be given a core to run on, and on a busy machine that wait can outlast the interval being measured — so the subtraction goes negative, not because anything arrived before it was sent, but because the stamp meant to mark the start was written late. Separately, the most widely used cross-broker benchmark for comparing brokers stamps its cross-process span in whole milliseconds and keeps only positive differences, so on a path faster than a millisecond it deletes samples at a rate fixed by arithmetic rather than chance. Neither failure appears in the output, and both are caught by checks that cost nothing.

The two failures share one number. Write T_{true} for the interval being measured and put beside it whichever timescale the instrument contributes, the scheduling stall before a stamp is written or the resolution of the stamp itself. When that instrument timescale exceeds T_{true}, the measurement stops describing the system and starts describing the instrument. Sub-millisecond broker paths are where that ratio crosses one for tooling in current use.

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie.

A. Contributions

1) **A deletion law for benchmark samples, established by manipulation** (Section IV). The OpenMessaging Benchmark admits an end-to-end sample only when a millisecond-quantized difference is positive, with no counter for what it discards, and over the full ledger of 223 runs the retained fraction falls to 0.0044%. Retention follows arithmetic the operator never sees: with send interval over quantum equal to p/q in lowest terms, retention can take only the $q{+}1$ values $0, 1/q, \ldots, 1$, and a run lands on one of the two that bracket T_{true}/τ. Fourteen arms across seven denominators behave as positioned, 9 of 10 powered arms reject a continuum null after correction, and a pre-registered payload manipulation moves an arm onto a grid vertex and off it.
2) **The inversion mechanism, established by manipulation on one clock** (Section V). An inversion occurs when the stall between an event and the clock read that labels it outlasts the interval being measured, so $\Pr[\text{inversion}] = \Pr[\text{stall} > T_{\mathrm{true}}]$. Real-time priority on the stamping threads at fixed utilization collapses the rate 7–$80\times$; identical utilization reached by two core geometries differs $2.07\times$; lengthening the true transport $77\times$ *lowers* the rate; an unfitted kernel trace predicts the observed rate to within a third. The span that inverts is referenced to an acknowledgment, and the send-referenced span over the same $738{,}730$ events never inverts, locating the fault in the reference stamp. Two brokers sharing neither client nor protocol invert at 8.67% and 8.19% on one clock: the fault follows the span, not the system under test.
3) **An audit, and the reporting rules that follow** (Sections III-B and VII-A). A sign check applied uniformly to all $2{,}266$ runs rejected $1{,}321$ of them, including every run behind our own first result. The practice is older than this paper: Paxson [1] treated physically impossible transit times as evidence of instrument failure and published the proportion of traces affected. Each half of the practice is established: pre-defined exclusion

with reported counts is standard statistical advice [2], physically impossible values are grounds for discarding a trace [3], and a standards body already voids a whole run on a clock criterion [4]. What we add is their conjunction — a physical-impossibility criterion, applied per run, with the campaign's rejection rate published as a headline quantity rather than a footnote — and the observation that the retention rate and the sign of each discard belong beside every reported latency.

B. Chronology, and what was withdrawn

We set out to compare two brokers on a live sports feed. The audit of Section III-B rejected our own first answer — a complete, statistically significant result set — and a later audit of the benchmark refuted a claim we had already made about it. What follows is what survived; the order of discovery, the withdrawn claims and the failed pre-registrations are in Supplement S35, because an audit whose own failures are hidden is worth less than one whose are not.

II. Background and Related Work

The claim that measurement can dominate a benchmark is not new; what is missing is the practice of checking for it run by run.

A. Streaming benchmarks

Broker comparisons appear continuously, in peer-reviewed venues and in the gray literature practitioners read. Apache Kafka and Redis Streams are the pair they most often name, and the OpenMessaging Benchmark [5] is the community's shared instrument. That experimental setup can dominate a result is established [6], and how to report one is a settled literature we follow [7], [8].

The stream-processing benchmark literature has already thought carefully about where to stand a clock. Karimov et al. [9] separate event-time from processing-time latency and keep the driver off the machines under test; Van Dongen and Van den Poel [10] use a single broker and its append timestamps "to ensure correct latency measurements", which is our one-clock rule arrived at independently. Neither work asks what a millisecond stamp does to a sub-millisecond path, or checks the sample the instrument keeps. Those two questions are this paper.

B. Cross-process time measurement

Lamport [11] established that a distributed system has no single physical time, only a causal partial order. That order decides this paper's central span: an acknowledgment at the producer and a record at the consumer are two branches from the broker's append, so neither precedes the other. Agreeing clocks is a well-known engineering problem [12], and instruments built to beat it exist: Cloudprofiler [13] calibrates cross-node counters to $\sim 92\,\mu$s, Lancet [14] stamps in the NIC and tests its samples for convergence, and P4STA [15] stamps in the switch dataplane. The timer's own accuracy is itself a measured quantity [16].

One prior result bounds our claim closely. Villain et al. [17], profiling FreeBSD against a DAG hardware reference, found the outgoing socket timestamp "does not respect causality" — packets leave before the stamp meant to mark their departure is written — under every stress pattern including none, and attributed host latency to interrupt processing, scheduling and context switching. That is Mode A's physical primitive, published in 2012 and measured more precisely than we measure it; we add its consequence for a two-process measurement, a law relating the inversion rate to the interval measured (Equation 3), and manipulations separating scheduling from its rivals. The audit half has its own ancestor: Paxson [1] treated physically impossible transit times as instrument failure and published the proportion of traces affected. Lancet is the nearest existing gate, and the difference is the one that matters: it adapts within a run and reports what it has, with the results attached and marked inconclusive, where we decline to publish at all — and neither it nor anything else we found reports the fraction of runs a campaign lost.

The observable is familiar in production, and the field has already argued about it. Tracing systems ship clock-skew adjusters that "modif[y] start time and log timestamps for spans that appear to be 'off' with respect to the parent span" [18]; an issue titled *Clock Skew Adjuster considered harmful* objected that the result is "wrong data and really hard to diagnose" [19], and Jaeger has defaulted the adjustment off since 2020. That conclusion was reached without a mechanism. Where the displacement is a late stamp rather than a skewed clock, correcting it moves evidence rather than error — and what nobody had was a measurement of how much evidence.

Concurrent work reaches the same observable by another route. In a 2026 preprint, Sharma et al. [20] count negative timing spans in distributed inference systems and propose a `causality_health` check; injecting skew, they see no violations up to 3 ms and clear ones by 5 ms, and read that as a skew threshold. We contradict a premise of that reading rather than their measurements. They hold that "queueing alone cannot produce negative timing spans"; we observe inversion rates near 23% at 88% utilization with no skew to blame, on one host and with an inter-host offset of 0.067 ms where two are used, and Section V moves that rate fiftyfold without touching a clock. Their own formulation is nearer ours than that premise allows — they note the minimum separation "exhibits variability due to scheduling jitter", with violation probability rising as ϵ exceeds its quantiles, which is Equation 3 with skew where we put the stall. Scheduling is a third channel, so a threshold in milliseconds of skew does not transfer to a loaded machine. The same attribution has been made about the benchmark we audit: asked why publish latency could exceed end-to-end latency, a maintainer answered that the stamps come from different nodes and "the clock are typically (in best case) skewed by ~100ms" [21]. The reporter replied that both processes ran on one node, and was told the measurement was then reliable. That is the arm our audit rejects most often.

C. Resolution and discarded samples

That a timer coarser than the interval invalidates it is textbook [16]. Metrology has long carried both our failure modes in one uncertainty budget: the NIST stopwatch guide lists the operator's reaction time beside the display resolution, and notes that at short intervals the resolution becomes significant against the instrument's rated accuracy [22]. We claim neither the pairing nor its novelty. That periodic sampling commensurate with a periodic phenomenon sees only part of it is stated in the IPPM framework [23], and RFC 3432 makes a randomized start mandatory for periodic streams [24]. Coordinated omission [25] is the mirror image: there the instrument fails to record samples that would have been slow, here it deletes samples that were fast, and Tene's point — that omission correlated with the value measured skews the result — covers both directions. The precondition has been seen before in another benchmark: YCSB reported sub-millisecond operations as zero milliseconds until it adopted microsecond histograms [26], and it kept those samples rather than deleting them. Millsap and Holt state the same arithmetic for interval timers and draw the opposite conclusion from it [27]: an event shorter than the resolution measures either one tick or zero according to whether it crosses a tick boundary — our Equation 4 — and coarse timing survives this in aggregate *because* the two signed errors cancel. A guard that admits only positive samples removes one of the two signs, and with it the cancellation the defence depends on; Supplement S41 sets this beside the GUM's rectangular model and the oldest statistical statement that discarding zeros is not free.

The mathematics of the interaction is older still, and it is not ours. Counter metrology settled it for time-interval averaging half a century ago, and Hewlett-Packard's Application Note 162-1 sets out all of it [28]. Its retention identity, $\Pr[R = Q{+}1] = F$ for an interval of $Q + F$ clock periods, is our Eq. 4. Its *class* of a repetition rate is our denominator q, from the same co-prime condition. The ceiling it derives on any gain in resolution, a factor $1/q$, is our Eq. 5, and it names $q = 1$ as the corner where averaging cannot help at all. It even lists the symptom to look for: a counter reading that appears to hang on a round number. Its cure is the one we arrived at independently, phase jitter on the repetition rate.

What is new is the sign of the operation. HP *keeps* both readings, and the retained fraction *is* the estimator: averaging is unbiased precisely because the $Q{+}1$ readings are counted alongside the Q readings. The benchmark discards that population. Our contribution is therefore not the law — we derived it, then found it in a counter manual — but its consequence under deletion: a harness released in 2018 throws away the quantity a 1970 counter measured with.

a) Sources of scheduling delay: A thread that is runnable does not always run. Work-conservation violations are documented for CFS-era schedulers [29]; ours is an EEVDF-era kernel. Queueing gives the leading-order account of waiting, and predicts correctly that utilization alone does not fix a waiting time — pooling and geometry matter at fixed ρ — so the geometry contrast of Table III refutes a single-parameter law rather than queueing itself. A 2026 preprint by Chandrasekar and Kramberger [30] applies exactly such a single-parameter M/G/1 framing to benchmark bias, which we adopt as a leading-order account and, on the load axis, ultimately refute (Section V). On a second axis they reach our mechanism's inflation half concurrently and independently: their client's own interpreter lock "artificially inflates" the latencies it reports, charging the instrument's scheduling to the server under test. What a positive number cannot show — the inversion, and the sign channel that makes the displacement legible — is what remains ours. Tail behavior under load [31] is the phenomenon our stall distribution samples, and that work also shows real-time priority collapsing a tail that renicing leaves alone — on the system under test, where we apply the same lever to the instrument.

III. Method

Everything below is measured on one clock unless the text says otherwise, and the spans we audit are named explicitly, because which two stamps are subtracted is the whole question.

A. Testbeds, and what the measurement proxies

Every finding comes from four Oracle Cloud `VM.Standard.E5.Flex` instances (three brokers, one driver), Ubuntu 22.04 on kernel `6.8.0-1057-oracle` (EEVDF scheduler), CPython 3.10, a private network, client libraries pinned identically, and `chrony`-disciplined clocks logged every minute. Measurement perturbs, and one early phase perturbed enough to be unusable: it pinned the load generator to a subset of cores while utilization was measured across all of them, diluting ρ. It is excluded from every result here, and the campaigns that remain measure utilization with no core pinning. A second testbed, a single workstation running the brokers under containers, produced our withdrawn first result and is reported only as an audit outcome.

a) The measurement at a glance: The harness runs producer and consumer as separate *processes on one host*, both stamping the wall clock (`time.time_ns()`, CLOCK_REALTIME), not a monotonic counter or the TSC, since a cross-process span needs a shared epoch that no per-core cycle counter provides. The cost is worth naming: CLOCK_MONOTONIC guarantees consecutive reads do not go backwards and CLOCK_REALTIME does not, so we forgo an exclusion the clock could have supplied and establish it from data instead (Section V). Nor did we record `current_clocksource`; the probe now does, too late for the runs reported here. The workload is a public football event feed (StatsBomb open data, CC BY-NC-SA 4.0) replayed against Kafka and Redis Streams. The send schedule is measured rather than assumed: pacer jitter is 67–69 μs at p90, and the achieved rate is recorded per run against elapsed wall time. The measured inter-host offset, where two hosts are used, is 0.067 ms. The primary measure is **Time-to-Insight** (TTI), from scheduled emission to availability at the consumer:

$$\mathrm{TTI} = \underbrace{(t_{\mathrm{send}} - t_{\mathrm{sched}})}_{\text{scheduling lag}} + \underbrace{(t_{\mathrm{recv}} - t_{\mathrm{ack}})}_{\text{transport proxy}} + \underbrace{\text{residual}}_{\text{broker-internal}} . \quad (1)$$

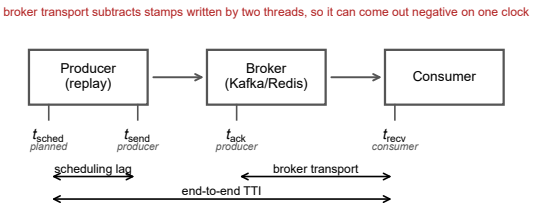

Fig. 1. **The four timestamps, and the span each metric covers.** Scheduling lag is measured inside one process. Broker transport subtracts a producer-side stamp from a consumer-side one, so it can come out negative when either process is delayed — which is the failure Section V takes apart.

The residual is the broker's own append-to-deliver interval, which no stamp of ours brackets; it is carried because TTI is a total, and named because an unnamed span is not a latency.

b) One component is a proxy, not a chain: This distinction is load-bearing. The transport term of Equation 1 subtracts the producer's receipt of the broker's acknowledgment from the consumer's receipt of the record. The broker's append precedes both of those events, but neither of them precedes the other: they are two branches of one cause. The acknowledgment stamp is therefore a *late, producer-side observation* of a broker-side event, and when the acknowledgment branch runs slower than the delivery branch the difference is negative without anything physically impossible having occurred. By contrast $t_{\mathrm{recv}} - t_{\mathrm{send}}$ and TTI itself are genuine chains: the producer sends, the broker appends, the consumer receives. Section V-A reports all four spans, and they behave completely differently.

B. The consistency check

We fix the rule before the final campaign and apply it to every run, including those whose results looked reasonable. **A run is rejected if more than 1% of its events carry a negative value in any latency component, or if the median of any component is negative.** A condition is usable only if all of its runs survive; where partly rejected we report the retention rate. The one-per-cent figure is a threshold, not a discovery: the supplement sweeps it from 0 to 20% and no condition behind our first result is usable anywhere in that range, so the withdrawal does not turn on where the line was drawn.

The justification is not that a negative is impossible, because for the transport proxy it is not. It is that a negative proves the reference stamp cannot serve as the origin of that event's interval. A run whose reference is unusable on more than one event in a hundred cannot report a latency, whatever the cause. The check is a *necessary* condition, not a validation procedure: a corpus that fails is unusable, and one that passes has cleared the cheapest available bar and nothing more. We use the sign as a gate rather than as an estimator, although the same constraint supports estimation, since offset and skew can be recovered from a delay envelope [1]. That was deliberate: an estimator corrects a record, a gate declines to publish from a run whose instrument demonstrably failed, and after this check destroyed our own first result we preferred declining.

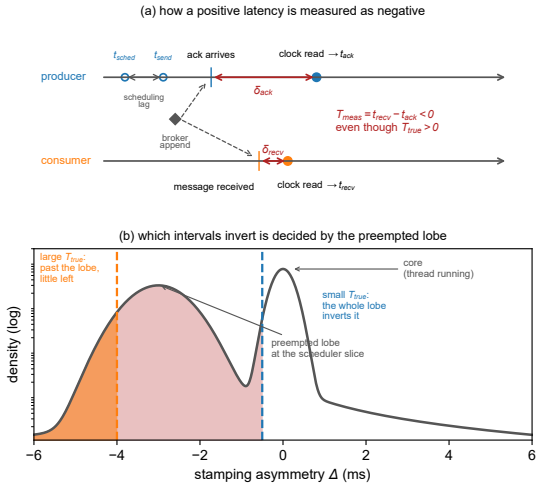

Fig. 2. (a) How a positive latency is measured as negative, on one clock: each side stamps some interval after its event, and when the producer's stamping delay exceeds the true delivery plus the consumer's delay, the difference inverts — the record is not early, the acknowledgment stamp is late (Section III-A). The producer's own stamps, t_{sched} and t_{send}, do precede the acknowledgment, and the span referenced to them never inverts (Table II). (b) Why the check bites hardest on small effects: the stamping asymmetry Δ has a narrow core when the stamping thread is running and a lobe at -3 ms, the scheduler's base slice, when it is not, so a small T_{true} is inverted by the whole lobe and a large one only by what lies beyond it.

C. A model of the failure

Figure 1 names the four stamps and the span each metric covers. Every timestamp is taken some interval after the physical event it claims to mark, because the thread that reads the clock must first be scheduled. This is not the cost of the clock call, which is small and separately measurable [16]; it is the delay before the call runs at all. Writing δ for those stamping delays,

$$T_{\mathrm{measured}} = T_{\mathrm{true}} + \Delta, \qquad \Delta \equiv \delta_{\mathrm{recv}} - \delta_{\mathrm{ack}}, \tag{2}$$

so Equation 2's difference inverts exactly when $\Delta < -T_{\mathrm{true}}$, giving

$$\Pr[\text{inversion}] = F_{\Delta}(-T_{\mathrm{true}}). \tag{3}$$

Equation 3 says the most useful thing in this paper (Fig. 2b). **Inversion probability is a property of the quantity being measured, not only of the machine**: a fixed distribution of Δ overlaps a small T_{true} almost entirely and a large one hardly at all, which is why our network-delay arm, transport in tens of seconds, passes every run while same-hardware arms measuring one millisecond fail wholesale.

D. Statistics

Every proportion below is a count over a stated denominator with a Wilson score 95% interval, Wilson rather than Wald because at the real-time arms' rates the normal approximation puts the lower bound below zero. Every ratio between two proportions carries a two-proportion z with pooled variance, and we state whether the intervals are disjoint. Location differences use Hodges–Lehmann shifts with distribution-free intervals and rank-sum tests; equivalence claims use TOST with the

margin stated and the interval printed; Holm correction is applied within a named family, and the corrected value is the one the tables display; percentile bootstrap intervals cover statistics with no closed form. The grid-membership inference uses a permutation null, described where it is used. Regression intervals are Student-t at $n - 2$ degrees of freedom. Tail indices are estimated on the traced histogram by grouped maximum likelihood with a profile-likelihood interval and by an exceedance ratio with a Wilson interval, because the buckets are interval-censored and a slope through survival points is not an estimator; the grouped estimator on log-spaced bins is Virkar and Clauset's, which is the Hill estimator for binned data, and we use their bootstrap for goodness of fit [32]. All interval arithmetic is recomputed from the committed artifacts at build time by `scripts/stat_intervals.py` and `scripts/tail_index_traced.py`, and the tables reporting it are generated rather than transcribed.

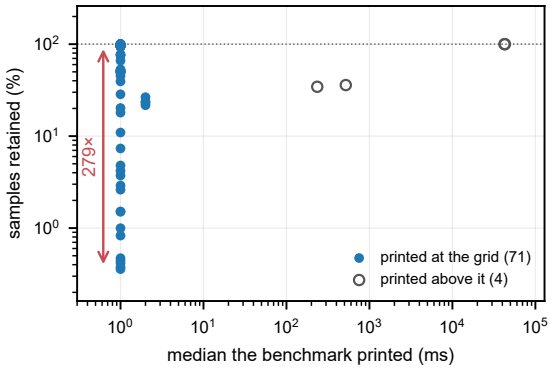

Fig. 3. **What the benchmark prints against what it kept in order to print it**, over every instrumented cell. At the quantum the printed median carries no information about retention: 71 cells report 1.0 or 2.0 ms while the retained fraction spans 279×. Away from the quantum the harness behaves, which is the point — the failure is confined to intervals shorter than the instrument.

IV. Mode B: A Benchmark That Deletes Its Own Samples

The cross-broker benchmark we audit computes its reported latency distribution from a subset of the samples it collected, does not record how large that subset was, and the size of the subset is determined by the send rate. Everything measured below is the OpenMessaging Benchmark in embedded mode, where driver and workers run in one JVM; its distributed mode is discussed in Section VII-C.

A. The guard, and its origin

In `LocalWorker.java`:294 the end-to-end latency is `now - publishTimestamp`, where `now` is read in the consumer and `publishTimestamp` is Kafka's producer-set `CreateTime`, itself a millisecond value. That difference is then promoted, `TimeUnit.MILLISECONDS.toMicros(now - publishTimestamp)`, so the guard in `WorkerStats.java`:95 — `if (endToEndLatencyMicros > 0)` — tests a variable named for microseconds that can only ever hold integer multiples of a thousand of them. The message is counted as received a few lines earlier, but a non-positive latency is dropped and no counter records how many.

The guard's origin is documented and its reasoning is sound. It was added in 2018 to stop negative values reaching the histogram, on the ground that end-to-end latency "is calculated based on time difference of `System.currentTimeMillis()` on two different machines" and "can be negative" [33]; the recorder is an `HdrHistogram Recorder`, which rejects negative values outright. Filtering for positives is the reasonable local fix, defensive rather than evasive. Its non-local consequences are two, neither considered at the time. First, `> 0` also deletes every sample that computes to exactly zero, and because `System.currentTimeMillis()` has millisecond resolution, any delivery faster than one millisecond does compute to zero; on co-located paths, where our own transport measures 0.1–0.5 ms, that is most of them. Second, the one observation which would prove the instrument had failed is the exact observation the guard removes. The convention is institutional rather than incidental: the survey of Section IV-D finds the same disposal, uncounted, in 4 independent tools.

The guard's scope is the most interesting thing about it. It is on the end-to-end path, which stamps in milliseconds; the same harness measures its publish span with a nanosecond clock and records every sample unconditionally, needing no guard because that span is taken inside one process. The coarse clock and the filter both landed on the one span that crosses processes.

This harness has lost data silently twice before, in defects that were found and fixed (Supplement S16). The guard is not one of those: it is a design choice, which is why it is still there.

a) The consequence, measured: Of the 75 instrumented cells whose own summary we captured, 71 report a median of 1.0 or 2.0 ms, and across those the benchmark computes its reported distribution from between $\mathbf{0.36}\%$ and $\mathbf{100}\%$ of the samples it takes. Two replicates of one cell — 200 B payload at 500 msg/s, same host — kept 431 and 120,434 samples, a 279-fold difference, and both reported a median of 1.0 ms with nothing in the output to distinguish them. Three uninstrumented runs print p50, p95, p99 and max all equal to 1.0 to three decimal places. The full ledger holds 223 instrumented runs, 11,532,492 discarded samples and 41,403 negatives, and over it the retained fraction falls as low as 0.0044% — four samples kept of ninety thousand. Figure 3 puts the two quantities on one pair of axes: the printed median is pinned at the quantum while retention moves across two and a half orders of magnitude, and the only cells that escape are the four whose payload is large enough that the quantum stops binding.

This is the discarded fraction that time-interval averaging uses as its measurement [28]. A counter that averages recovers the sub-tick part of the interval from how often the reading comes out one count high; the fraction is the signal. Reporting a distribution conditioned on that fraction, without reporting the fraction, discards the estimator and keeps the residue.

B. Phase, and the grid it produces

Publish latency across these cells takes only two values, 0.3 and 0.4 ms, while retention varies 279-fold. A two-valued predictor cannot account for a range like that whatever its correlation, and the correlation it does have (+0.31 over 71 cells) carries the sign Equation 4 predicts. What moves retention is where the producer's send instants fall against the millisecond grid. A sample survives when its delivery crosses a tick boundary, which for a delivery shorter than one tick happens with probability T_{true}/τ:

$$\text{retention} \;=\; \min\!\left(1,\; \frac{T_{\text{true}}}{\tau}\right) \qquad \text{(uniform phases).} \qquad (4)$$

Both incommensurate arms sit near 50%, implying $T_{\text{true}} \approx 0.5$ ms at $\tau = 1$ ms, which agrees with our own unquantized transport measurement.

a) Commensurability is not binary: Phases are not uniform when the send interval is commensurate with the tick. Write the send interval over the tick in lowest terms as $\Delta/\tau = p/q$ with $\gcd(p, q) = 1$. After q sends the phase returns to its start, so a producer visits exactly q distinct phases. Retention is then a count out of q, so it can only take the $q{+}1$ values $0, 1/q, \ldots, 1$, and a single run lands on one of the two that bracket T_{true}/τ: $q = 1$ is the all-or-nothing corner, an incommensurate rate is effectively continuous, and everything between is a grid, coarser meaning less reproducible. The replicate spread follows:

$$\text{spread} \;=\; \begin{cases} 100/q & \text{when } T_{\text{true}}/\tau \text{ sits mid-cell,} \\ 0 & \text{when } T_{\text{true}}/\tau \text{ sits on a grid point.} \end{cases} \qquad (5)$$

Supplement S23 reports every arm. A round number is the worst possible choice: 500 msg/s is exactly 2.000 ms, $q = 1$, and its four replicates retain 0.47, 1.51, 18.02 and 99.99 per cent. At 457 and 383 msg/s, incommensurate, replicates hold to 2.1 and 2.8 points despite intervals differing by 19%. The exactness matters: 889 msg/s sits 0.00014 from 9/8 and behaves as fully continuous, while 300 msg/s spreads 30.5.

C. The grid as inference

A table of spreads is not a test, so we state one. For each arm let D be the mean distance from its replicates to the nearest vertex of the q-grid, normalized by the cell half-width. Under the continuum null, retention is a continuous function of rate and replicate positions are uniform on the cell; the null distribution is obtained by permuting replicate positions within the arm (10^4 permutations, so the smallest attainable p is 2.5×10^{-4}). Table I reports every arm with $n \geq 5$ and prints the corrected value, not the raw one. 9 of the 10 powered arms reject the continuum null at $\alpha = 0.05$ after Holm correction within the family of 12. The tenth, 900 msg/s, rejects before correction ($p = 0.044$) and not after ($p = 0.131$); we report it as unresolved rather than as support. The remaining 2 arms, 400 and 800 msg/s, have no power by construction: when T_{true}/τ sits on a vertex the grid prediction and the continuum prediction coincide, so no test can separate them. A model that claimed separation everywhere would be the more suspicious one. Figure 4 shows why the set matters more than any one arm: every powered arm falls below the diagonal, and the two that do not are the two the model predicts cannot.

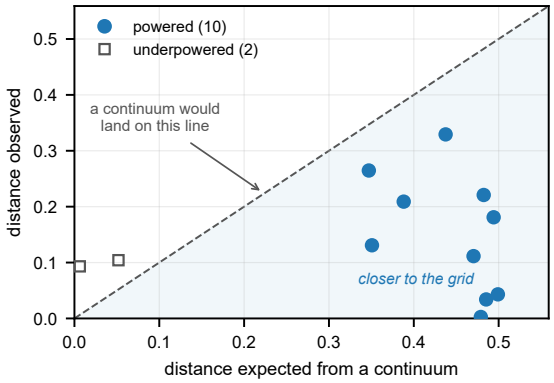

Fig. 4. **Grid membership, as a set.** Each arm's observed mean distance to the nearest grid vertex against the distance a continuum would give. Points below the diagonal sit closer to the grid than chance allows; the 2 open markers are the arms where the two hypotheses coincide and no test has power. Those two sit slightly above it, which is what no power looks like: where a vertex nearly coincides with the continuum expectation, the normalized distance is replicate noise and its sign carries nothing. Per-arm corrected p-values are in Table I.

TABLE I

GRID MEMBERSHIP AGAINST A CONTINUUM NULL. D IS THE MEAN NORMALIZED DISTANCE TO THE NEAREST GRID VERTEX; UNDER THE NULL ITS EXPECTATION IS THE VALUE IN THE *null* COLUMN. p_{Holm} IS A PERMUTATION p-VALUE (10^4 DRAWS) AFTER HOLM CORRECTION WITHIN THE FAMILY OF ALL 12 ARMS; THE CORRECTED VALUE IS WHAT DECIDES THE VERDICT. ARMS MARKED *flat (coincident)* ARE THOSE THE MODEL PREDICTS FLAT, WHERE THE TWO HYPOTHESES MAKE THE SAME PREDICTION AND THE TEST HAS NO POWER. THIS TABLE IS GENERATED FROM `GRID_MEMBERSHIP.CSV` AT BUILD TIME.

rate	p/q	n	D	null	p_{Holm}	verdict
1250/s	4/5	5	0.131	0.351	0.003	grid
1000/s	1/1	5	0.003	0.479	0.003	grid
900/s	10/9	5	0.265	0.347	0.131	not resolved
875/s	8/7	5	0.209	0.388	0.003	grid
800/s	5/4	5	0.104	0.052	1.000	flat (coincident)
700/s	10/7	5	0.329	0.438	0.009	grid
625/s	8/5	10	0.112	0.470	0.003	grid
600/s	5/3	6	0.034	0.485	0.003	grid
500/s	2/1	5	0.043	0.499	0.003	grid
400/s	5/2	5	0.093	0.006	1.000	flat (coincident)
300/s	10/3	10	0.221	0.482	0.003	grid
250/s	4/1	5	0.181	0.494	0.003	grid

a) A pre-registered confirmation: Payload moves T_{true}, hence the grid position, and Equation 5 dictates the arm's class with nothing else changed. The prediction was registered before the runs, and it came true (Fig. 5): spread collapses to 13.6 at 32 KB, three replicates within 0.06 of each other, pinned near the 2/3 vertex, and re-opens to 26.7 at 64 KB, the flat–full boundary crossed in both directions by one manipulated variable. One registered detail failed: the 32 KB pin sits at 69.2, not the predicted 66.67.

A comprehensive overnight campaign ran 63 cells at $n \geq 5$, with six predictions and their falsifiers registered in advance; every cell completed validly. One prediction was confirmed, three falsifiers fired, one was not evaluable and one split. A cumulative drift account is falsified by duration invariance: intermediate counts of 1, 1 and 0 over one, three and ten

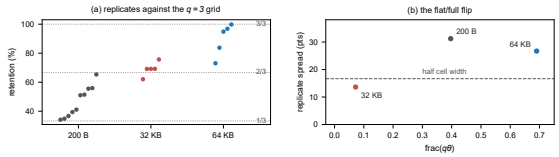

Fig. 5. The campaign's confirmed prediction. (a) Retention replicates at 300 msg/s ($q = 3$) by payload, against the thirds grid: 200 B straddles both branches, 32 KB pins near the $2/3$ vertex, 64 KB re-opens to the upper cell. (b) Replicate spread against $\mathrm{frac}(q\theta)$: the flat–full boundary at half the cell width is crossed in both directions by the one manipulated variable.

minutes, with vertices pinned throughout, so what stands is per-send jitter rather than accumulation. Incommensurate arms at 1053 and 1219 msg/s measure 47.2 and 48.2%, refuting the linear extrapolation one registered prediction assumed.

D. Generality, and the sign channel

To show the law is not an artifact of one JVM, we reproduced the guard in an independent Python harness sharing no code with the benchmark. On one clock it recorded **zero** negative differences across 905,040 samples, as a causal chain requires, while the grid behavior reappeared intact. Across two `chrony`-disciplined hosts retention wandered from 13.4 to 27.0% over four replicates where the same arm on one clock held to 0.8 points: cross-host, the deleted fraction couples to the run-time synchronization state, and nothing in the output records either.

Nor is the disposal confined to this harness. We audited 7 tools at source across 4 languages and 7 vendors, classifying each finding with the code that classified the benchmark. Apache Pulsar's performance client computes a millisecond cross-host difference and admits it only when positive, with no counter [34] — a second vendor reaching the same design independently, which makes this a pattern rather than one project's oversight. Two tools do something worse than filter: `fio` returns zero for a negative interval in each of three timing functions, under a comment reading "time warp bug on some kernels?" [35], and a block-tracing tool substitutes one nanosecond (Supplement S36). Substitution leaves nothing missing from the output, so retention computed downstream reads 100% and a value never measured enters the distribution. A fourth disposal needs two files to see: `wrk2` tests its interval, prints a panic block asserting a negative can never happen, and records it anyway, unchecked; its *vendored* copy of HdrHistogram then either trips a live assertion or refuses the derived index with a `false` nobody reads (Supplement S36). Neither file is wrong alone, and on either path the sample is never recorded and nothing counts the loss. In all, 5 of the 7 dispose of the sample without counting it. Two are not: Rezolus counts its discards in a metric whose description names the cause [36], which shows the rule is practical, and `emqtt-bench` computes the same millisecond cross-process difference and keeps every sample, its `> 0` test guarding a counter rather than the histogram [37] — we report it because we first read it as a filter and it is not, and a guard is only a deletion if what it guards is the sample.

Separating the discards by sign is what makes the guard's cost legible. Across all 223 runs the Kafka-driver corpus's 10,913,263 discarded samples contain **not one negative**: the most negative value at any load, size or rate is $0\,\mu$s. Those discards are resolution failures, not clock failures — a distinction an unsigned counter cannot make. The same channel then caught the real thing in the Redis-driver replication: 41,403 negatives, every one exactly $-1000\,\mu$s, the minimum the quantum can express, with 41,397 of them in a single run where they were *half of all samples taken*, beside six kept.

That value is not a coincidence, and the driver explains it. The Redis driver takes its publish timestamp from `entry.getID().getTime()` in `RedisBenchmarkConsumer.java`:72 — the stream-entry identifier the *Redis server* assigns on `XADD` [5] — whereas the Kafka driver uses a producer-set stamp. The Redis end-to-end span is therefore a difference between two millisecond-floored clocks on two hosts, and a sub-tick offset between two floored clocks can produce exactly one negative value, $-1000\,\mu$s, and no other. That is precisely what the corpus contains. It also explains the contrast that makes the sign channel legible: the Kafka-driver span floors two stamps whose difference is taken inside one JVM, and it never goes negative at all. Clock discipline was clean at every logged minute, which is the point — this failure needs no misbehaving clock, only two of them and a floor. That guard hides this as well as your fastest measurements. Nor do the drivers share a measurand: Kafka's span is client-to-client on one clock, Redis's server-to-client across two (Supplement S43).

V. Mode A: Why a Latency Comes Out Negative

The negatives in our own corpus are produced by scheduling, and we establish this by moving scheduling while holding everything else fixed.

A. What the audit rejected

The rule of Section III-B rejects 862 of 1,382 runs on the workstation testbed (62.4%) and 459 of 884 on the cloud testbed (51.9%), every run behind our first result among them. At saturation Kafka's median transport proxy reads -6.4 ms at one hundred connections.

Table II separates the components. The acknowledgment-referenced proxy is negative on 62,264 of 738,730 events (8.43%; Wilson intervals are under 0.1 points here and omitted hereafter), and 3,976 runs exceed the one-per-cent threshold on it. The send-referenced span and TTI, which are causal chains, are negative on **no event at all**. Nothing in this corpus arrived before it was sent; the reference stamp was late. The rejections are therefore evidence of an unusable reference rather than of impossible physics, which is precisely what the gate is for.

a) Why it looked correct: The rejected result was monotone in concurrency, significant after correction ($p = 9.0 \times 10^{-11}$), and passed every conventional check we knew: replicate counts, effect sizes, corrected significance, sanity plots. The acknowledgment timestamp is read only once the reply has arrived and its waiting thread is rescheduled, so under

TABLE II
NEGATIVES BY SPAN AND BROKER (5,913 RUNS, 738,730 EVENTS, ONE CLOCK). ONLY THE ACKNOWLEDGMENT-ORIGIN SPAN INVERTS, AND IT DOES SO UNDER BOTH BROKERS: KAFKA ON 8.67% OF EVENTS, REDIS ON 8.19%. THE CAUSAL CHAINS ARE CLEAN UNDER EACH BROKER, SO THEIR PER-BROKER COLUMNS ARE DASHED RATHER THAN REPEAT A ZERO THREE TIMES.

Span	Origin stamp	Kafka	Redis	All
$t_{\text{recv}} - t_{\text{ack}}$	acknowledgment	31,899	30,365	62,264
$t_{\text{recv}} - t_{\text{send}}$	send (chain)	—	—	0
$t_{\text{out}} - t_{\text{send}}$	send (chain)	—	—	0
TTI	emission	—	—	0

saturation the consumer has already recorded receipt; the send stamp is read before the message leaves and cannot be late that way, whichever client runs (Supplement S43). That is a timestamping race under load, which is why medians stay positive at moderate load and invert wholesale only at saturation.

b) What the check cannot catch: A great deal. Load-induced inflation that stays positive is physically possible and needs a better instrument to detect. Our own second withdrawal is the example: a twentyfold end-to-end gap that was a per-run start-up cost read as a per-event constant, entirely causally consistent and entirely wrong. Causal consistency is necessary, not sufficient.

B. A rare state, not a wide distribution

The stamping thread is either running, in which case the stamp is late by a narrow jitter of width σ_c, or preempted and off-CPU. Writing $p(\rho)$ for the probability of the second state and $S(\cdot)$ for the residual's survival function,

$$\Pr[\text{inversion} \mid T_{\text{true}}] \;=\; p(\rho)\; S(T_{\text{true}}). \tag{6}$$

Load changes how *often* the thread that must record the time is not running, not how wide the ordinary jitter is. Going from idle to the knee multiplies the fitted core width σ_c by 5 and the inversion rate — the mass of Δ beyond $-T_{\text{true}}$ — by 61. The rate has a ceiling below one: fully saturated it reaches 0.37, so even then most events are stamped either unpreempted or by a thread whose stall is shorter than the interval being measured.

C. The experiments that settle it

No pair of arms in Fig. 6 overlaps. Real-time priority on the stamping threads moves occupancy while utilization stays fixed (Table III, top). The rate falls by factors of 39 and 54; the Wilson intervals of the two arms are disjoint at both load levels ($z = 19.8$ and $z = 31.9$), and both residual rates land on the floor the model names in advance, $C_0 \approx 0.004$. *This refutes every account in which the inversion rate is a function of utilization*, including the queueing form we ourselves adopted: ρ did not change and the rate changed fiftyfold. Across eight matched pairs spanning 60 to 95% load the factor ranges 7–80×.

Three further manipulations agree. At $k = 6$ both load geometries sit at $\rho = 0.7531$, identical to four decimals, and the inversion rates differ by 2.07× ($z = 10.3$), reproduced at

TABLE III
THE MECHANISM, BY MANIPULATION (2,985 MATCHED EVENTS PER CELL; WILSON 95% INTERVALS IN BRACKETS). TOP: STAMPING THREADS MOVED TO `SCHED_FIFO` WITH BACKGROUND LOAD UNCHANGED; ACHIEVED UTILIZATION MATCHES TO 0.001 BETWEEN ARMS, SO OCCUPANCY ALONE MOVED. BOTTOM: IDENTICAL UTILIZATION REACHED BY TWO CORE GEOMETRIES ($\rho = 0.7531$ IN ALL FOUR k=6 ARMS); A FUNCTION OF ρ RETURNS ONE VALUE FOR ONE INPUT.

Manipulation	Arm	Rate	95% CI	Factor (z)
Priority, 75%	ordinary	0.1320	0.1203–0.1446	
	real-time	0.0034	0.0018–0.0062	39× (19.8)
Priority, 88%	ordinary	0.3049	0.2886–0.3216	
	real-time	0.0057	0.0036–0.0091	54× (31.9)
Geometry, original	concentrated	0.0824	0.0731–0.0928	
	spread	0.1709	0.1578–0.1848	2.07× (10.3)
Geometry, repl.	concentrated	0.0600	0.0520–0.0691	
	spread	0.1229	0.1117–0.1352	2.05× (8.4)

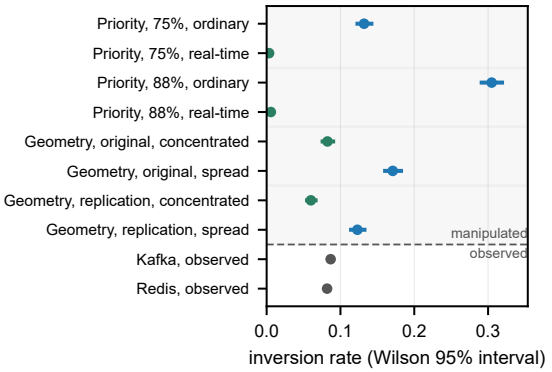

Fig. 6. **Every manipulation separates; the broker does not.** Wilson 95% intervals. Above the rule, the four matched pairs of Table III, utilization held fixed within each: what moves the rate is where the stamping thread sits, not how loaded the machine is. Below it, observed rather than manipulated, the two brokers of Table II; their intervals are narrower than the plotted point.

2.05× by a full replication. Multi-server queueing expects geometry to matter at fixed ρ, so this will not surprise a queueing-literate reader; what it rules out is the single-parameter form we had adopted and pre-registered, where ρ alone fixes the rate. Payload padding lengthened transport 76.9× while the inversion rate *fell* 4.1×, monotonically, at a 0.012 utilization spread: the direction Equation 3 predicts and the opposite of a stress account. Finally, measured with `runqlat` on the `sched_wakeup` and `sched_switch` tracepoints [38], filtered to the harness processes and sharing no instrument, estimator or data with the application-level rate, the probability that a run-queue stall outlasts the interval agrees with the observed inversion rate across 3 arms at two load levels, ratios $0.78, 1.06, 1.32$ with no consistent sign: an application-level failure rate predicted to within a third, unfitted. A probe on every context switch is not free, and we measured what it costs by running the same cell untraced: the inversion rate is 0.272 untraced against 0.231 traced ($z = 3.6$). The comparison above is against the traced arm's own rate, so the agreement is between two instruments observing one machine; the untraced rate is the one to quote for the machine itself.

a) The load-axis predictions, and their failure: The knee sweep placed conditions where the candidate load laws di-

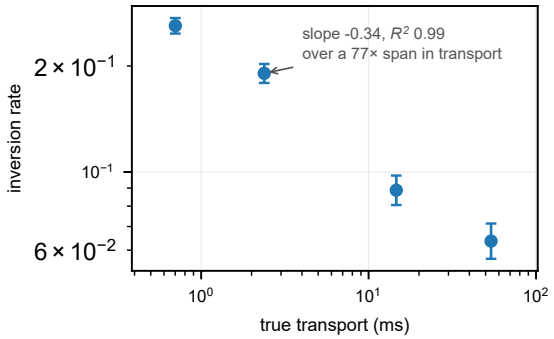

Fig. 7. **A longer interval inverts less often.** Inversion rate against true transport over the payload sweep, Wilson 95% intervals. The same stall distribution overlaps a short interval almost entirely and a long one hardly at all, which is Equation 3 as an experiment: lengthening what is being measured lowers the rate, where a load-driven account predicts no change.

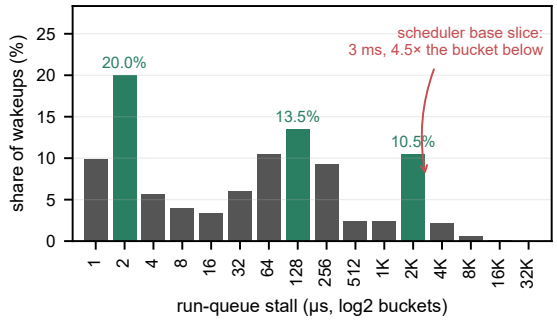

Fig. 8. **The traced run-queue stall distribution is trimodal.** 551,956 wakeups, `bpftrace` $\log_2$ buckets, modes shaded. The jitter core sits at a few microseconds and a second mode at 128–256 μs; the third, holding 10.5% of all wakeups, lands on the scheduler's derived base slice of 3 ms. A power law would have to descend monotonically across this axis.

verge. The M/G/1 form fitted *worse than the mean* ($R^2 = -0.05$ against a fitted exponential's 0.93), and our own registered bracket missed a 1.44× growth by predicting 2.45–3.07×. Both were parameterized in ρ, and ρ is not the variable. The Pollaczek–Khinchine form remains motivation rather than fitted result.

D. The stall spectrum, and the scheduler's slice

That this distribution is not a single heavy tail is known: Gregg reported it as bimodal in the post that introduces `runqlat`, the tool we use here [38]. What we add is which modes, and why the upper one sits where it does.

Raising the payload from 0 to 256 KB lengthens the true transport 77×, and over that span the inversion rate falls monotonically with T_{true} (Fig. 7); log–log least squares over the four payload levels returns an effective exponent of 0.339 (0.234–0.443, Student-t at two degrees of freedom). The fit is in Supplement S32; four points do not earn an equation here. An earlier version read the kernel trace as an independent confirmation of it; we withdraw the claim, because estimating the traced index properly is what refuted it.

Over the 551,956 traced wakeups the grouped maximum-likelihood index on 0.25–2 ms is 1.19 (1.18–1.20) against an exceedance index on 0.5–2 ms of 0.21 (0.20–0.21). Two estimators of one quantity cannot differ sixfold unless the model is wrong, and a bootstrap goodness-of-fit agrees: $p < 0.0004$ over 2,500 replicates drawn from the fitted power law itself. The reason is visible without fitting: the counts do not decrease monotonically, as a power law must: they have 3 local maxima, and the last is a *mode* at 2–4 ms holding 10.5% of all wakeups, 4.5× its lower neighbor. A count against Gregg's modes would not be like for like — different host, workload, scheduler — but neither this share nor this position has, to our knowledge, been attributed to a scheduler constant before. Above it the survival is well behaved and light — grouped maximum likelihood beyond 4 ms gives 2.04 (2.00–2.07), a finite variance. Figure 8 is the histogram itself, and the shape is the argument: no monotone curve passes through it.

That mode is the preempted state of Equation 6, at a scale the maintainers had already named: with run-to-parity a task's lag is held within "$[-(\text{slice} + \text{tick}) : (\text{slice} + \text{tick})]$" [39]. The histogram adds that this is not merely a worst case but where a tenth of all wakeups land. The constants are *derived*, not measured, from the published kernel package and the campaign's own data: each geometry condition loads k cores and records the achieved utilization, so k/ρ recovers 8 six times over with a spread of 0.05; the kernel's rule gives a base slice of $4 \times 0.75\,\text{ms} = 3$ ms, which is the maintainers' own arithmetic — "0.75 msec * (1 + ilog(ncpus)) . . . 3.00 for 8 cpus and above" [40] — and is specific to this kernel, the constant having become 0.70 ms in v6.15, after our 6.8; and the build sets `CONFIG_HZ`=1000, a 1 ms tick. With run-to-parity a woken thread that loses the wakeup-preemption check waits for the incumbent to exhaust that slice, and expiry is noticed at the tick. That tick is also why the mode is a band and not a spike: the same posting notes a task "can only wait until the next tick to consider that it has reached its deadline, and will run 1ms longer". A thread descheduled at the wrong instant therefore reads its clock about a slice late, which on a path whose true transport is 0.1–0.5 ms is six to thirty times the interval being measured. No scheduler tunable is written anywhere in the archived campaign, which is why the derived value is the one that applied — strong evidence rather than proof, and reported as such. A mean over this distribution is dominated by a mode the operator never sees, so a mean-based counter cannot flag this failure.

VI. What the Brokers Actually Do

Once the failed runs are removed, the broker comparison that motivated the work is small, and one configuration choice dwarfs it.

On gated artifacts the brokers sit within a millisecond of each other on the transport proxy: a Hodges–Lehmann shift of 0.408 ms (90% interval 0.389–0.419), TOST rejecting both one-sided nulls against a 1 ms margin at $p < 0.001$, holding between 0.408–0.417 ms across 3 concurrency levels. That span is not a causal chain; TTI, which is, is equivalent over the same levels against a wider margin (Supplement S6). It is not a purchasing argument.

One condition reverses the ordering, and it is not the broker. One-way delay injected identically at both leaves

Kafka tracking it almost additively while Redis collapses to a median transport of 4.6 s at 5 ms of injected delay, 31.3 s at 20 ms and 102.5 s at 50 ms over five feeds per arm — 900 to $2{,}050$ times the delay applied. The cause is the consumption pattern: a per-message acknowledgment turns each event into a sequential round trip, where a prefetching consumer serves from buffer. A falsifier registered in advance confirms it — batching those acknowledgments two hundred at a time cuts the median from $4{,}138$ ms to 103 ms, replicated four times. Both settings are invisible on a co-located testbed — the round trip they multiply is too short to matter — so the standard evaluation environment hides the choices that matter most.

VII. Discussion

Both failures are cheap to detect and neither is detected, so the remedy is a reporting rule rather than a better clock.

A. For benchmark authors

Report a physical-consistency audit. Any cross-process timestamp difference admits a sign check that costs nothing, and every run it rejected here had passed every conventional check we knew. Better instruments exist [13], [14], [15]; whatever the instrument, check its output against causality and publish the rejection rate, as network clocks have for thirty years [1], [3]. The discipline is tested only by a campaign the rate costs something, and ours cost us a result.

Name the span you are subtracting. A difference of two stamps is a latency only if the first event causally precedes the second, and an acknowledgment at the producer does not precede a record at the consumer (Section III-A). We caught our own instance only on counting the send-referenced span separately (Table II).

Count your events before you quote a percentile. A median over as few as seven events per run is not a property of the system, and it survives a hundred runs: a deterministic per-run artifact reproduces beautifully.

Make the timestamping path unpreemptable, and say that you did. The one mitigation our measurements support directly: real-time priority on the stamping threads cut the inversion rate 7 to $80\times$ at unchanged background load, and busy-polling a dedicated core should buy the same for a burned core. Two caveats: the floor is not zero, so the check stays; and a real-time or spinning stamping thread changes the system it measures, so where that is unacceptable, report retention instead.

Dither the send instant, and publish the retention rate. Randomizing probe timing to avoid synchronizing with a periodic phenomenon is long-standing advice [23], [24]; what we add is why — the phenomenon synchronized with is the timestamp quantum itself. Publish beside the latency the retention rate, the timestamp resolution and the synchronization state, none recoverable afterwards: a summary from 0.36% of samples is typographically identical to one from all. Neither half of the rule is unprecedented — OWAMP requires a packet stamped in the future to be recorded unaltered within its timeout window [41], ETSI requires removed measurement cycles to be reported [42], and RFC 7679 treats negative singletons as usable distribution data [43]. The sign is the channel: negative means the reference stamp failed, computes-to-zero means the resolution failed, and one unsigned total cannot distinguish them. Supplements S39 and S40 give both precedents in full.

Record the achieved rate, not the flag meant to produce it. Verify it per run against elapsed wall time: the send schedule is an experimental variable in Mode B, not a nuisance parameter, and we found runs where configured and achieved rates disagreed.

B. The limits of a better clock

Hardware-timestamped PTP [12] would take two LAN hosts from our $\approx$70 μs of `chrony` agreement to sub-microsecond, but behind a one-millisecond stamp, 70 μs and 70 ns produce byte-identical records: the guard deletes the same samples and the grid law is unchanged. The remedies are therefore ordered: record at native resolution, count what is discarded, dither the send instant, and only then does synchronization quality become the frontier — our own cross-host retention wandered 13.4–27.0% under clocks agreeing to well under a tick (Section IV-D). Designs that remove the one-way requirement predate the clouds needing it [1], [44], each resting on an assumption its users rarely state (Supplement S37).

The obvious redesign does not help either: a consumer-to-producer acknowledgment removes skew but not this failure, and it measures a producer-side round trip rather than staleness at the consumer (Supplement S43.6).

a) Resolution, as distinct from synchronization: Finer stamps fix the lesser half. A microsecond stamp makes $T_{\mathrm{true}}/\tau \gg 1$ here, so Equation 4's deletions disappear and the grid recedes to paths faster than the new quantum — but the positivity guard survives, and genuine negatives still occur at microsecond resolution: stamping stalls (Section V), the $\approx$70 μs inter-host offset, virtualized clocks. The guard deletes them silently too, converting an uncounted *resolution* failure into an uncounted *instrument* one.

C. Threats and limitations

A negative span proves something is wrong; attributing it to scheduling is an inference, separated from its rivals by manipulation, with the eliminations listed at the end of Section V. Beyond that list, the clustering of consecutive late wakes ($z = -6.9$ at the co-located floor, at every load including none) is the signature of a shared scheduling delay, not independent kernel granularity. `chrony` reports the inter-host offset of Section III-A as comfortably sub-tick, but bounds its own error at 4–7 ms per host — a pair may disagree by 12 ms against a 1 ms stamp — so the cross-host channel stays open rather than ruled out. It did not produce the Redis-driver negatives, which the driver's own stamp accounts for exactly (Section IV-D), but it is why we claim no cross-host bound in general.

a) Scope, and the alternatives eliminated: Three widely used runtimes document this structure without drawing its consequence for a cross-process subtraction (Supplement S38).

The mechanism requires a stamping path that can be preempted. Designs that pin a thread to a dedicated core and busy-poll, or that stamp in hardware, remove the state Equation 6 depends on, and we make no claim about them. Within our own setting the alternatives are each eliminated: clock skew (one clock by construction on the rejecting arms; zero negatives in $905{,}040$ two-clock samples); cross-CPU incoherence under migration, below; utilization (identical ρ, rates twofold apart); timer noise (`SCHED_FIFO` at fixed utilization collapses the rate 39–$54\times$, and noise does not respect scheduling class); stress (transport $77\times$ longer *lowers* the rate); and direct observation (a `sched_switch` histogram predicts the rate to within a third, unfitted). One rival is not the scheduler's: the stamp is a CPython call, so it waits on the interpreter lock too, invisibly to a tracepoint probe — bounded because that kernel-only estimator brackets the observed rate $(0.78, 1.06, 1.32)$ instead of under-predicting (Supplement S43.4).

The clock-incoherence channel needs its own answer, because our machines are virtual and a hypervisor need not give a guest a counter coherent across its vCPUs [45]. The clocksource went unrecorded and the instances are reclaimed, but the probe caught the smallest increment the clock could report, 90 ns, and read cost separates the candidates by an order of magnitude: that leaves `kvm-clock` or `tsc`, both of which the kernel uses only where cross-CPU coherence is asserted or checked (Supplement S42). The corpus closes the channel again on headroom, not on shared endpoints. Both audited spans are producer-to-consumer differences, so a clock-domain discrepancy displaces both alike; only the headroom differs. An offset able to explain the deepest inversion, -99.8 ms, would drive the send-referenced span about that far below zero — yet over $5{,}913$ runs it never goes negative, and its lowest value is $+220\,\mu$s, the smaller of the two brokers' floors; the other is $+805\,\mu$s. That floor bounds any such offset at $455\times$ too small; in the $4{,}549$ runs where the acknowledgment span inverts, the send span stays clean in 100%, over an identical event set. So on one host two clock reads cannot disagree by more than $220\,\mu$s without a measured span going negative that never does — an order of magnitude tighter than `chrony` claims for itself cross-host (Section VII-C).

The mechanism's constants are those of one EEVDF-era kernel (6.8); CFS-era schedulers, the regime Lozi et al. document, may shift them. The deletion law is demonstrated on one benchmark and reproduced in one independent harness; the pattern it requires — a quantized stamp, a paced producer, a positivity guard — is common, and Section IV-D finds the disposal in 5 tools, though every one of those is a reading of source rather than a measured deployment. Nor is the guard confined to the tool we measured: the same expression appears unchanged in more than a dozen public forks and in tooling released in 2026, so the exposure is current. The OpenMessaging distributed mode failed identically on every attempt, so that tool's cross-host exposure rests on source reading and our own two-host harness rather than on running it. Exposure in a given deployment depends on its path latency and producer rate, which is why we recommend publishing a retention rate as routine practice rather than calling any published number wrong.

VIII. Conclusion

A latency benchmark measures the system only while the instrument is faster than the thing being measured. Below that line the two failures reported here appear, invisible in the output: a stamp written late by a descheduled thread inverts the interval it was meant to time, and a stamp quantized to a millisecond, filtered for positivity, deletes most of a sub-millisecond path by arithmetic the operator never sees. Neither needs a better clock, and neither is expensive to detect. Check the spans whose endpoints are stamped in different places, count what your instrument throws away, and publish the retention rate beside the number.

Acknowledgment

The author thanks R. Duvignau, whose reading established that the acknowledgment-referenced span is not a causal chain and prompted the recount that reorganized this paper; D. Gregg, for the readability critique that shaped the previous version and for the self-consistency argument in Section IV; J. Kunkel, for the same-clock protocol contrast, the offset-estimation route and the measurement-model figure; N. Herbst, for load-generation validation and the timer-characterization literature; M. Luthra, for the kernel-tracing pointers; M. Kogias, for the probe-placement argument; A. Ousterhout, for the preemptibility scope condition; and L. Tratt, for the question about timestamp resolution answered in Section IV.

In accordance with IEEE policy, Anthropic's Claude assisted with code development and verification, manuscript editing, and literature search. Experimental design, scientific claims, and interpretation are the author's, and all reported quantities are reproducibly computed from the committed data and archived code.

Artifact Availability

Code, analysis and manuscript are archived at 10.5281/zenodo.22044877, the measurement dataset at 10.5281/zenodo.22044891. Every number here is recomputed from the committed data by the archived code at build time, and the test suite fails if the text and the data disagree.

References

[1] V. Paxson, "On calibrating measurements of packet transit times," in *Proc. ACM SIGMETRICS*. ACM, 1998, pp. 11–21.

[2] B. Krishnamachari, "How to do statistical evaluations in ECE/CS papers: a practical playbook for defensible results," arXiv preprint arXiv:2605.00428, 2026, section 18, "Outliers, failed runs, and exclusion rules".

[3] V. Paxson, "Strategies for sound Internet measurement," in *Proc. ACM Internet Measurement Conf. (IMC)*. ACM, 2004, pp. 263–271, physical impossibility as grounds for discarding a trace.

[4] Standard Performance Evaluation Corporation, "SPECjbb2015 run and reporting rules, v1.02," 2018, §3.4.5; an initial clock offset beyond ten minutes voids the run.

[5] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/; accessed 19 August 2026.

[6] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009, pp. 265–276.

[7] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, 2007, pp. 57–76.

[8] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2015, pp. 73:1–73:12.

[9] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proc. IEEE 34th Int. Conf. Data Engineering (ICDE)*, 2018, pp. 1507–1518.

[10] G. Van Dongen and D. Van den Poel, "Evaluation of stream processing frameworks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1845–1858, 2020.

[11] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[12] IEEE, "IEEE standard for a precision clock synchronization protocol for networked measurement and control systems," IEEE Std 1588-2019, 2020, revision of IEEE Std 1588-2008.

[13] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, "Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads," *arXiv preprint arXiv:2205.09325*, 2022.

[14] M. Kogias, S. Mallon, and E. Bugnion, "Lancet: A self-correcting latency measuring tool," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2019, pp. 881–896, §4.3: terminates and reports the results with an inconclusive verdict.

[15] R. Kundel, "P4STA: High precision network function benchmarking," in *Accelerating Network Functions Using Reconfigurable Hardware*, ser. Springer Theses. Springer Nature Switzerland, 2024, pp. 93–115.

[16] M. Kuperberg, M. Krogmann, and R. H. Reussner, "Metric-based selection of timer methods for accurate measurements," in *Proc. 2nd ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, Mar. 2011, pp. 151–156.

[17] B. Villain, M. Davis, J. Ridoux, D. Veitch, and N. Normand, "Probing the latencies of software timestamping," in *Proc. IEEE Int. Symp. Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, 2012, pp. 1–6, dTrace breakdown against a DAG hardware reference; reports software stamps that "do not respect causality" under load.

[18] Jaeger contributors, "`model/adjuster/clockskew.go`," https://github.com/jaegertracing/jaeger, 2026, `model/adjuster/clockskew.go` at v1.57.0; adjusters run in the query service, and the default has been off since v1.20.0. Read 2026-08-21.

[19] ——, "Clock skew adjuster considered harmful (issue #1459)," Issue tracker, 2019, https://github.com/jaegertracing/jaeger/issues/1459. Read 2026-08-21.

[20] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, "Time, causality, and observability failures in distributed AI inference systems," *arXiv preprint arXiv:2604.21361*, Apr. 2026, open Compute Project, Unified Intelligent Infrastructure workstream.

[21] OpenMessaging Benchmark contributors, "Can the average publish latency be larger than end-to-end latency? (issue #216)," https://github.com/openmessaging/benchmark/issues/216, 2021, the anomaly attributed to inter-node skew, on a same-node report.

[22] J. C. Gust, R. M. Graham, and M. A. Lombardi, "Stopwatch and timer calibrations (2009 edition)," National Institute of Standards and Technology, NIST Special Publication 960-12, 2009, uncertainty budget carrying both operator reaction time and display resolution.

[23] V. Paxson, G. Almes, J. Mahdavi, and M. Mathis, "Framework for IP performance metrics," Internet Engineering Task Force, RFC 2330, May 1998.

[24] V. Raisanen, G. Grotefeld, and A. Morton, "Network performance measurement with periodic streams," Internet Engineering Task Force, RFC 3432, Nov. 2002.

[25] G. Tene, "How NOT to measure latency," 2015, talk, Strange Loop / QCon; coined "coordinated omission".

[26] M. Wiederhold, "Sub-millisecond latencies are reported as taking 0 milliseconds (issue #41)," Yahoo! Cloud Serving Benchmark, Oct. 2011, https://github.com/brianfrankcooper/YCSB/issues/41; fixed by pull request #42 and released in 0.1.4.

[27] C. Millsap and J. Holt, *Optimizing Oracle Performance*. Sebastopol, CA: O'Reilly, 2003, section 7.7, "Quantization Error".

[28] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766; undated; the year is its *HP Journal* citation.

[29] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, "The Linux scheduler: a decade of wasted cores," in *Proc. 11th European Conf. Computer Systems (EuroSys)*. ACM, 2016, pp. 1–16.

[30] A. Chandrasekar and J. Kramberger, "Identifying and mitigating systemic measurement bias in production LLM inference benchmarks," *arXiv preprint arXiv:2605.24217*, 2026.

[31] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, "Tales of the tail: Hardware, OS, and application-level sources of tail latency," in *Proc. ACM SoCC*. ACM, 2014.

[32] Y. Virkar and A. Clauset, "Power-law distributions in binned empirical data," *The Annals of Applied Statistics*, vol. 8, no. 1, pp. 89–119, 2014.

[33] S. Guo, "Avoid adding negative metric values (pull request #56)," OpenMessaging Benchmark, commit ebb5d6b8, Mar. 2018, https://github.com/openmessaging/benchmark/pull/56; accessed 18 August 2026.

[34] Apache Pulsar contributors, "`PerformanceConsumer.java`," Source repository, 2026, https://github.com/apache/pulsar; `PerformanceConsumer.java`. Read 2026-08-21.

[35] J. Axboe, "fio: Flexible I/O tester — `gettime.c`," Source repository, 2026, https://github.com/axboe/fio; the guard is in three timing functions. Read 2026-08-20.

[36] IOP Systems, "Rezolus: systems performance telemetry — `scheduler_discarded_samples`," Source repository, 2026, https://github.com/iopsystems/rezolus; counts samples discarded for out-of-order timestamps. Read 2026-08-20.

[37] EMQ Technologies, "`emqtt-bench`: MQTT benchmark tool — `src/emqtt_bench.erl`," Source repository, 2026, https://github.com/emqx/emqtt-bench; the guard is on the counter, not the sample. Read 2026-08-21.

[38] B. Gregg, "Linux bcc/BPF run queue (scheduler) latency," https://www.brendangregg.com/blog/2016-10-08/linux-bcc-runqlat.html, Oct. 2016, introduces `runqlat`; reports the distribution as bimodal without attributing a mechanism.

[39] V. Guittot, "`sched/fair`: Manage lag and run to parity with different slices," Linux kernel mailing list, Jul. 2025, states the base-slice protection and the slice-plus-tick lag bound.

[40] zihan zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel mailing list, commit 2ae891b82695, 2025, reviewed-by V. Guittot; reduces the constant to `0.70` in v6.15.

[41] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, "A one-way active measurement protocol (OWAMP)," Internet Engineering Task Force, RFC 4656, Sep. 2006.

[42] ETSI, "Speech and multimedia transmission quality (STQ); QoS aspects for popular services in mobile networks; part 4: Requirements for quality of service measurement equipment," European Telecommunications Standards Institute, Technical Specification TS 102 250-4 V2.3.1, Nov. 2019, clause 5.1, General requirement for data logging.

[43] G. Almes, S. Kalidindi, M. Zekauskas, and A. Morton, "A one-way delay metric for IPPM," Internet Engineering Task Force, RFC 7679, Jan. 2016, obsoletes RFC 2679. Section 3.5 keeps negative values as distribution data rather than discarding them.

[44] Y. Geng, S. Liu, Z. Yin, A. Naik, B. Prabhakar, M. Rosenblum, and A. Vahdat, "Exploiting a natural network effect for scalable, fine-grained clock synchronization," in *Proc. 15th USENIX NSDI*. USENIX Association, 2018, pp. 81–94.

[45] Linux Kernel Documentation, "KVM-specific MSRs: `MSR_KVM_SYSTEM_TIME_NEW`," https://docs.kernel.org/virt/kvm/x86/msr.html, 2026, flag bit 0, CPUID `0x40000001` bit 24. Read 2026-08-21.

Gustavo Pedro Ricou received the B.Sc. degree in physics from the Universidade do Porto, Portugal, and specialized in nuclear fusion science at the University of Stuttgart, Germany, and the Université de Lorraine, France. He received the Higher Diploma in computing from Atlantic Technological University and an M.Sc. in software design with cloud-native computing from the Technological University of the Shannon, Ireland, both with first-class honors, and is currently pursuing an M.Sc. in statistics and data science at Trinity College Dublin, Ireland. He is a Software Engineer with BoscaSports, building real-time pipelines for live sports. His research concerns measurement validity — what a reported number is evidence of, and when a routine convention distorts what it summarizes — in computer systems, environmental measurement and sports analytics, using pre-registration and physical-consistency auditing.